



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

ОТЧЁТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:
«Оптимизирующий компилятор подмножества
Фортрана»

Студент _____
(Группа)

(Подпись, дата)

В. А. Шемякин
(И.О. Фамилия)

Руководитель НИР

(Подпись, дата)

А. В. Коновалов
(И.О. Фамилия)

2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические основы оптимизации циклических вычислений	5
1.1 Архитектурные предпосылки оптимизации и проблема памяти	5
1.2 Циклы как основной объект анализа компилятора	6
1.3 Информационные зависимости по данным	7
1.4 Математическое моделирование: пространство итераций . .	9
1.5 Базовые преобразования вложенных циклов	10
1.5.1 Перестановка циклов	10
1.5.2 Блочное разбиение	10
1.5.3 Скачивание пространства итераций	11
1.5.4 Метод гиперплоскостей	12
1.5.5 Раскрутка циклов	12
1.5.6 Слияние и расщепление циклов	12
1.6 Взаимодействие цикловых преобразований и порядок их применения	13
2 Методы ускорения гнезд циклов итерационного типа	14
2.1 Определение и специфика гнезд циклов итерационного типа	14
2.2 Моделирование информационных зависимостей и вычисление векторов расстояний	15
2.3 Алгоритм построения матрицы скачивания	16
2.4 Блочное разбиение и оптимизация порядка обхода внутри тайла	17
2.5 Аналитическая модель выбора оптимальных размеров тайлов	18
2.6 Дополнительные скалярные оптимизации линейных участков	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22

ВВЕДЕНИЕ

В задачах вычислительной математики и физического моделирования значительное время тратится на обработку массивов. При этом важным фактором является не только число арифметических действий, но и порядок, по которому программа обращается к данным. На практике вложенные циклы задают основную нагрузку и определяют характер работы с памятью. При обработке цикла происходит постоянный обмен данными между регистрами, кэш-памятью и оперативной памятью.

В связи с этим проявляется проблема «стены памяти»: арифметические устройства часто простаивают в ожидании операндов. В результате снижается скорость выполнения программы. Поэтому оценка численной программы не сводится к арифметической сложности. Порядок обхода пространства итераций приходится менять так, чтобы уже загруженные данные использовались несколько раз, а независимые участки вычислений становились видимыми для компилятора.

В данной работе рассматриваются гнезда циклов итерационного типа. Они возникают в конечно-разностных схемах, методах релаксации, клеточных автоматах и других расчетах, где значение точки сетки пересчитывается по значениям соседних точек. Обычно внешний цикл задает номер итерации или временного слоя, а внутренние циклы обходят пространственную сетку. Для такого класса программ строится алгебраическая модель пространства итераций; далее к ней применяются скачивание, блочное разбиение (тайлинг) и метод гиперплоскостей.

Цель данной работы состоит в детальном теоретическом обосновании методов оптимизации многомерно вложенных циклов и в их адаптации для применения в архитектуре оптимизирующего компилятора подмножества языка Фортран.

Для достижения поставленной цели в работе решаются следующие задачи:

- рассмотреть аппаратные ограничения, из-за которых порядок обхода массивов становится существенным для производительности;
- описать формальную модель пространства итераций и зависимостей по данным;
- выделить преобразования, необходимые для работы с итерационными гнездами циклов: перестановку, скачивание, блочное разбиение и метод гиперплоскостей;

- обосновать последовательность применения этих преобразований в компиляторе подмножества Фортрана;
- получить модель выбора размеров блоков с учетом емкости и структуры кэш-памяти.

1 Теоретические основы оптимизации циклических вычислений

1.1 Архитектурные предпосылки оптимизации и проблема памяти

Производительность процессорных ядер и подсистемы памяти росла разными темпами. Ядра стали выполнять больше операций за единицу времени, но доступ к оперативной памяти ускорился заметно медленнее. В численных программах узкое место поэтому часто возникает не в арифметическом блоке, а на этапе доставки данных. Такое расхождение принято называть проблемой «стены памяти».

На современных процессорах сложение двух чисел с плавающей точкой выполняется за один или несколько тактов, тогда как обращение к оперативной памяти занимает на порядки больше времени. Этот разрыв частично скрывает многоуровневая кэш-память: L1, L2 и L3. Ближайший к ядру уровень быстрее, но меньше по объему; более дальние уровни вмещают больше данных, однако отвечают медленнее [1]. Регистры остаются самым быстрым хранилищем, но их мало, и порядок использования данных внутри цикла приобретает практическое значение.

Кэш не устраняет проблему доступа к данным автоматически. Выигрыш появляется тогда, когда программа вскоре снова обращается к уже загруженным элементам либо последовательно читает соседние адреса. При большом шаге обхода массива строка кэша используется неполностью, а затем вытесняется до повторного обращения. В задачах с большими массивами добавляется еще один источник задержек — промахи буфера трансляции адресов, увеличивающие стоимость доступа к памяти.

Нужно учитывать и единицу обмена между уровнями памяти. Данные передаются строками фиксированной длины; в распространенных микроархитектурах такая строка занимает 64 байта. Поэтому обращение к одному элементу массива фактически загружает и соседние элементы. При последовательном обходе одна строка обслуживает несколько операций подряд. Если индексы меняются с большим шагом, значительная часть загруженных данных не участвует в вычислениях, и полезная пропускная способность памяти уменьшается.

Ассоциативность кэша задает еще одно ограничение. В k -ассоциативном L1-кэше при $k = 8$ адреса, попавшие в один набор, делят между собой восемь

позиций. Для многомерных массивов с неудачными размерами строк это приводит к конфликтам: отдельные строки вытесняются, хотя в других наборах кэша еще остается свободное место. При выборе формы блока приходится учитывать не только его общий объем, но и отображение адресов на наборы кэша.

Для оптимизирующего компилятора это означает необходимость работать не только с выражениями в теле цикла, но и с порядком выполнения итераций. В научных программах основное время обычно занимают циклы над массивами; порядок вложенности, размер блоков и направление обхода прямо влияют на пространственную и временную локальность [2]. Регулярный доступ помогает и аппаратному предварительному чтению: устойчивый шаблон обращений позволяет процессору заранее подкачивать данные в кэш.

Сходные требования возникают и при использовании векторных расширений набора команд (SSE, AVX, AVX-512 в семействе x86 и их аналоги в других архитектурах). Одна команда обрабатывает несколько элементов, но только при регулярной структуре внутреннего цикла. Нужны последовательное расположение данных и отсутствие зависимостей между соседними операциями. Поэтому преобразование гнезда должно, насколько это возможно, приводить внутренний цикл к однотипным независимым действиям над смежными элементами массива.

1.2 Циклы как основной объект анализа компилятора

Компилятор рассматривает цикл как участок программы, где один и тот же блок инструкций выполняется многократно [3]. Для Фортрана типичны счетные циклы с оператором DO. Их границы и число итераций во многих случаях известны до входа в тело цикла либо выражаются через параметры программы. Это делает анализ потока управления и обращений к данным более определенным, чем в циклах с произвольным условием выхода.

Для полиэдральных преобразований особенно удобны тесные гнезда циклов. Между заголовками вложенных циклов в них нет дополнительных операторов, а вычисления сосредоточены во внутреннем теле. Такое гнездо естественно описывается как многомерное пространство итераций: одной точке соответствует одно выполнение тела цикла. Если исходный фрагмент записан иначе, компилятор предварительно приводит его к каноническому виду, применяя распространение

присваиваний, удаление мертвого кода, расщепление циклов и сходные преобразования.

Для Фортрана традиционно важна семантика работы с массивами. Отсутствие неявного наложения адресов в параметрах подпрограмм дает возможность в большинстве случаев считать обращения к разным массивам независимыми. Благодаря этому повышается точность построения графа зависимостей, и компилятор получает больше свободы для преобразований. Тот факт, что многомерные массивы располагаются в памяти по столбцам, жестко задает направление последовательного доступа к данным. Опираясь на это, оптимизатор может подбирать наиболее удачный порядок обхода итераций, например, производить перестановку циклов.

Анализируя тело цикла, компилятор находит переменные, порождающие информационные связи. В первую очередь рассматриваются те вхождения, которые работают как генераторы (то есть пишут в память) или как использования (читают из памяти). Изучая такие обращения, можно понять, какие данные стоит приватизировать, а какие зависимости реально мешают распараллеливанию.

Чтобы формально описать гнездо циклов, обычно выстраивают математическую модель пространства итераций. При этом каждому вхождению массива ставится в соответствие аффинное отображение точек этого пространства в целочисленное пространство индексов массива. На практике индексные выражения сводятся к виду $\sum_{k=1}^n a_k i_k + c$, где a_k — целые коэффициенты, а c — константа. Такие аффинные функции нужны для вычисления векторов расстояний и построения решетчатого графа программы. Если обращения оказываются нерегулярными, например из-за косвенной адресации, их исключают из классического анализа зависимостей.

1.3 Информационные зависимости по данным

Теория распараллеливания циклов опирается на анализ информационных зависимостей [4]. Любое преобразование компилятора допустимо только при сохранении семантики исходной программы; на практике это означает, что зависимые инструкции должны выполняться в прежнем порядке.

Информационная зависимость фиксируется между двумя обращениями к памяти, когда они могут относиться к одной и той же ячейке и хотя бы одно

обращение выполняет запись. В анализе циклов обычно различают три типа таких зависимостей:

1. Истинная зависимость. Возникает в тех случаях, когда генератор S_1 записывает данные в ячейку, а следующее за ним использование S_2 их считывает. Такая зависимость отражает реальную передачу данных в алгоритме, поэтому устранить ее простым переименованием нельзя.
2. Антязависимость. Возникает, когда использование S_1 читает значение до того, как генератор S_2 его перезапишет. Такая связь часто устраняется приватизацией переменных.
3. Выходная зависимость. Возникает, когда два генератора, S_1 и S_2 , последовательно модифицируют одну и ту же ячейку памяти.

Для многомерных гнезд принято выделять циклически независимые (loop independent) и циклически порожденные (loop carried) зависимости. Вторые появляются, если обращения к одной ячейке происходят на разных итерациях. Тот цикл, на итерациях которого и возникает эта связь, называют ее носителем. Чтобы без ошибок распараллелить код или поменять порядок итераций, нужно для каждой дуги графа зависимостей четко определить массив таких носителей.

Когда зависимость оказывается циклически независимой (то есть уровень переноса нулевой), она никак не мешает распараллеливать внешние циклы. Но если возникают циклически порожденные связи, то приходится либо оставлять цикл-носитель последовательным, либо прибегать к трансформирующим преобразованиям — например, к скашиванию.

Иногда встречаются так называемые ложные зависимости. Они возникают из-за повторного использования переменной для хранения новых данных, хотя реальной передачи информации не происходит. Эта проблема решается переименованием или приватизацией таких переменных. Расход памяти при этом несколько возрастает, но становится возможным применение более сложных легальных преобразований, таких как тайлинг.

На практике встречается и другая ситуация: зависимость возможна по грубой оценке, но не возникает ни при одном допустимом наборе параметров. Консервативный анализ обязан считать такие обращения связанными. Более точный параметрический анализ решает системы целочисленных неравенств с параметрами и отделяет мнимые зависимости от реальных. После этого граф зависимостей стано-

вится менее ограничительным, а последующие преобразования получают больше свободы. Развитие этих методов сделало возможным автоматическое применение многомерных преобразований, которые раньше чаще выполнялись вручную.

1.4 Математическое моделирование: пространство итераций

Чтобы применить к задаче строгие математические методы, любое гнездо из n циклов удобно рассматривать как n -мерное целочисленное пространство итераций. При таком подходе каждая копия тела цикла ассоциируется с вектором счетчиков $I = (i_1, i_2, \dots, i_n)^T$. Сами границы циклов образуют систему линейных неравенств, которая в итоге задает множество допустимых точек — полиэдр в пространстве $\mathbb{Z}^n$ [5].

Если программа имеет регулярную структуру, ее вычисления можно наглядно описать через элементарные решетчатые графы. В этом случае для каждой дуги (u, v) в графе зависимостей берут разность векторов счетчиков для зависимых итераций: $I_{dst} - I_{src}$. Когда эта разность остается одинаковой для всех представителей дуги, ее называют вектором расстояния (distance vector) D .

Главное условие корректности последовательной программы — лексикографическая положительность векторов расстояний ($D \succ 0$). Любые преобразования компилятора, которые меняют пространство итераций умножением вектора счетчиков на матрицу T , обязаны это свойство сохранять. То есть новый вектор $D' = T \cdot D$ тоже должен быть лексикографически положительным. Только так можно гарантировать, что преобразованная программа работает эквивалентно исходной [6].

С позиций полиэдральной модели задача оптимизации заключается в поиске функции планирования, то есть другого порядка обхода точек, который сокращает время работы программы и сохраняет семантику алгоритма. Для анализа зависимостей применяются методы целочисленного линейного программирования. Они требуют значительных вычислительных затрат, поэтому на практике их используют в основном для регулярных гнезд небольшой глубины.

Линейные преобразования пространства задаются унимодулярной целочисленной матрицей T ($|\det T| = 1$), а их действие равносильно замене переменных $I' = T \cdot I$. Требование унимодулярности гарантирует, что объем и целочисленная структура пространства останутся неизменными. Поэтому задача оптимизатора

состоит в поиске матрицы T , которая удовлетворяет условию $T \cdot D \succ 0$ и обеспечивает приемлемый баланс между параллелизмом и локальностью данных.

1.5 Базовые преобразования вложенных циклов

Среди множества существующих преобразований наибольший интерес для ускорения итерационных алгоритмов представляют перестановка циклов, скашивание, тайлинг и метод гиперплоскостей [1].

1.5.1 Перестановка циклов

Перестановка циклов (Loop Interchange) меняет местами вложенность смежных циклов. Обычно это делают, чтобы улучшить локальность данных или подготовить код к векторизации. Поскольку в Фортране многомерные массивы хранятся по столбцам, внутренний цикл предпочтительно связывать с первым индексом массива. В этом случае обращения к памяти идут с единичным шагом и кэш используется эффективнее. Если порядок индексов нарушен, доступ к памяти становится разреженным, что увеличивает число промахов кэша.

Чтобы перестановка двух смежных циклов была корректной, достаточно проверить векторы расстояний: после обмена координат ни один из них не должен стать лексикографически отрицательным. Помимо улучшения пространственной локальности, такая перестановка часто позволяет задействовать SIMD-инструкции, поскольку внутренний цикл начинает обращаться к памяти с единичным шагом.

1.5.2 Блочное разбиение

Блочное разбиение, или тайлинг (Loop Tiling), применяется в первую очередь для улучшения временной локальности. Пространство итераций разрезается многомерными гиперплоскостями на небольшие блоки — тайлы. Из-за этого меняется принцип обхода: программа сначала выполняет итерации внутри одного тайла и лишь затем переходит к следующему [7].

На уровне кода появляются новые внешние циклы, которые перебирают сами тайлы, тогда как внутренние циклы проходят по точкам внутри выбранного блока. Размеры тайлов подбираются так, чтобы данные, необходимые для вычис-

ления блока, помещались в быстрый кэш L1 или L2. Это позволяет многократно переиспользовать элементы массива, уже загруженные из оперативной памяти, пока они не вытеснены новыми данными. При этом остается жесткое ограничение: тайлинг корректен только при неотрицательных компонентах векторов зависимостей. Если встречаются отрицательные компоненты, перед тайлингом необходимо выполнить скашивание.

На практике тайлинг может применяться иерархически в соответствии с многоуровневой структурой кэш-памяти. Внешние уровни тайлинга обеспечивают размещение рабочего набора в кэше L2 или L3, а внутренние — в кэше L1 и в регистровом файле. Подобный многоуровневый тайлинг требует согласованного выбора размеров блоков на каждом уровне и приводит к гнезду циклов, глубина которого в три-четыре раза превышает исходную. Дополнительная сложность сгенерированного кода компенсируется существенным приростом производительности: современные реализации умножения матриц демонстрируют эффективность, близкую к теоретическому пределу процессора, именно благодаря тщательно подобранному многоуровневому тайлингу.

1.5.3 Скашивание пространства итераций

Скашивание применяется в ситуациях, когда векторы зависимостей содержат отрицательные компоненты, препятствующие применению тайлинга или распараллеливания. Данное преобразование геометрически изменяет форму пространства итераций из ортогональной, прямоугольной, в параллелограммную [8].

Алгебраически скашивание эквивалентно умножению вектора итераций на нижнетреугольную унимодулярную матрицу. Скашивание с правильно подобранными коэффициентами сдвигает итерации внутреннего цикла пропорционально значениям внешнего цикла, и это делает отрицательные компоненты векторов зависимостей строго положительными. Поэтому скашивание чаще всего выступает как подготовительный этап для последующей оптимизации. Унимодулярность матрицы скашивания гарантирует биективность отображения пространства итераций, что сохраняет общее число выполнений тела цикла и не вносит избыточных вычислений.

1.5.4 Метод гиперплоскостей

В алгоритмах с перекрестными зависимостями, например в неявных методах решения уравнений в частных производных, прямое распараллеливание циклов обычно невозможно. Метод гиперплоскостей, или метод волнового фронта, выделяет параллелизм за счет иной организации обхода. Пространство итераций покрывается параллельными гиперплоскостями, а вычисления продвигаются от одной плоскости к другой. Точки, лежащие на одной гиперплоскости, не зависят друг от друга по данным и потому могут выполняться параллельно.

1.5.5 Раскрутка циклов

В отличие от преобразований пространства итераций, раскрутка относится к скалярным оптимизациям тела цикла. Тело дублируется несколько раз с соответствующей корректировкой индексов, а число итераций уменьшается. Это сокращает накладные расходы на управление циклом: сравнение счетчика и условные переходы. Кроме того, в развернутом теле появляется больше независимых инструкций, что полезно для планировщика процессора и автоматической векторизации. Коэффициент раскрутки выбирается осторожно: слишком большое тело усиливает давление на регистровый файл и может вытеснять инструкции из кэша первого уровня.

1.5.6 Слияние и расщепление циклов

Слияние объединяет несколько идущих подряд циклов с одинаковым пространством итераций в один общий цикл. Это позволяет сократить накладные расходы на инициализацию циклов и улучшает временную локальность, если оба цикла обращались к одним и тем же массивам данных.

Расщепление разбивает один большой цикл на несколько независимых. Эта трансформация полезна для снижения давления на регистровый файл, для предотвращения вытеснения данных из кэша инструкций, а также для изоляции фрагментов кода, которые содержат циклически порожденные зависимости, от фрагментов, которые могут быть легко векторизованы.

1.6 Взаимодействие цикловых преобразований и порядок их применения

Порядок применения преобразований имеет самостоятельное значение. Они не независимы: результат одного шага может открыть возможность для следующего или, наоборот, закрыть ее. Например, чрезмерная раскрутка внутреннего цикла мешает последующему тайлингу по этому же циклу, а слияние двух циклов способно создать новые зависимости и тем самым запретить перестановку. На практике используются два подхода: фиксированный порядок оптимизационных проходов, подобранный эмпирически, либо поиск последовательности преобразований внутри единой математической модели.

Полиэдральная модель соответствует второму подходу. Последовательность преобразований представляется одной унимодулярной матрицей, а ее элементы подбираются как решение задачи целочисленного линейного программирования. Целевая функция учитывает параллелизм, локальность и объем коммуникаций; ограничения отвечают за сохранение всех векторов зависимостей. Такой метод хорошо описывает задачу в целом, но быстро приводит к высоким вычислительным затратам. Для гнезд глубины 3–5 уровней поиск еще реалистичен, для более глубоких случаев обычно вводятся эвристические ограничения.

Альтернативный подход состоит в последовательном применении эвристических правил, которые подобраны экспертно. Типичная последовательность включает в себя: нормализацию циклов и устранение ложных зависимостей на предварительной фазе; применение перестановки для улучшения пространственной локальности; скашивание для устранения отрицательных компонент в векторах расстояний; тайлинг для улучшения временной локальности; метод гиперплоскостей для извлечения параллелизма; и, наконец, раскрутку и скалярные оптимизации на финальной фазе. Каждая фаза принимает локально оптимальное решение на основе доступной в этот момент информации, что не гарантирует глобальной оптимальности, но существенно упрощает реализацию компилятора и его сопровождение.

2 Методы ускорения гнезд циклов итерационного типа

2.1 Определение и специфика гнезд циклов итерационного типа

Рассматриваемый алгоритм относится к гнездам циклов итерационного типа — классу структур, который часто встречается в научных расчетах. К нему относятся многие разностные схемы, итерационные решатели систем линейных алгебраических уравнений и фрагменты программ математического моделирования физических процессов [9].

В таких гнездах уровни вложенности имеют разные роли. Внешний цикл (счетчик t или i_0) задает номер итерации численного алгоритма, временной слой или шаг сходимости. Внутренние циклы ($i_1, \dots, i_n$) обходят многомерную пространственную сетку по фиксированному расчетному шаблону.

Далее рассматриваются строго тесные гнезда циклов. В них генераторы, то есть левые части операторов присваивания, имеют канонический вид $a_{i_1-c_1, \dots, i_n-c_n}$, а использования в правой части — $a_{i_1-d_1, \dots, i_n-d_n}$. Константы c_k и d_k задают форму расчетного шаблона. Счетчик внешнего итерационного цикла i_0 в индексные выражения массивов явно не входит: предполагается работа с несколькими буферами либо изменение матрицы на месте при переходе между временными слоями.

У этого класса гнезд есть как удобные для оптимизации свойства, так и жесткие ограничения. Фиксированный расчетный шаблон и регулярные индексные выражения позволяют вычислять векторы зависимостей явно; их число конечно и не зависит от размеров сетки. Границы циклов при этом задают прямоугольный полиэдр итераций. Но зависимости по временному измерению нельзя устранить переименованием или приватизацией: они выражают саму суть итерационного алгоритма и передачу данных между соседними временными слоями.

Формальное описание удобно строить через решетчатый граф: его вершины соответствуют точкам пространства итераций, а дуги — информационным зависимостям между представителями вхождений переменных. В тесном гнезде все вычисления находятся во внутреннем теле, поэтому зависимость может переноситься внешним итерационным циклом, одним из пространственных циклов или их комбинацией. При построении матрицы скашивания важно знать именно этот уровень переноса, а не только сам факт зависимости.

В гнездах итерационного типа зависимости описываются компактно: достаточно знать постоянные смещения в индексных выражениях. Если генератор и использование обращаются к одному массиву с разными смещениями, вектор расстояний получается непосредственно из этих констант. Полный перебор точек пространства итераций в таком случае не требуется; компилятор работает с конечным набором дуг графа зависимостей.

Выбор оптимизации зависит от расчетного шаблона. В алгоритме Якоби значение точки на новом слое использует только соседние значения предыдущего слоя; внутри одного временного шага пространственные зависимости отсутствуют, и пространственные циклы распараллеливаются относительно просто. В алгоритме Гаусса–Зейделя часть соседних значений уже обновлена на текущем слое. Отсюда возникают пространственные зависимости с отрицательными компонентами, и прямое распараллеливание становится некорректным. В этой ситуации скашивание и метод гиперплоскостей извлекают параллелизм из алгоритма, который в исходной записи выглядит последовательным.

Для гнезд итерационного типа характерна высокая доля обращений к памяти. В типичном разностном шаблоне одно обновление элемента требует 4–9 чтений и одной записи, тогда как число арифметических операций имеет тот же порядок. Программа становится чувствительной к пропускной способности памяти: процессор может выполнить арифметику быстрее, чем получить операнды. Тайлинг улучшает ситуацию за счет повторного использования уже загруженных данных и приближает исполнение к возможностям целевой архитектуры.

2.2 Моделирование информационных зависимостей и вычисление векторов расстояний

Оптимизирующий компилятор начинает с построения модели информационных зависимостей. В гнезде итерационного типа размерности $n + 1$ дуга (u, v) появляется между генератором и использованием, если оба обращения относятся к одному участку памяти.

В пространстве $\mathbb{Z}^{n+1}$ такой зависимости соответствует разность векторов счетчиков циклов, при которых выполняются связанные обращения. Внешняя итерация продвигается вперед по времени, поэтому первая координата, связанная с циклом i_0 , положительна и обычно равна 1. Остальные координаты берутся из раз-

ностей констант-смещений в индексных выражениях. Для дуги (u, v) используется формула:

$$\text{dist}(u, v) = (1, d_1 - c_1, d_2 - c_2, \dots, d_n - c_n). \quad (1)$$

Если хотя бы один вектор расстояний содержит отрицательную координату среды $d_k - c_k$, прямой тайлинг применять нельзя. При наивном разбиении может оказаться, что блок, который должен быть вычислен раньше другого, расположен после него в лексикографическом порядке. Тогда порядок зависимых вычислений нарушается, и преобразованная программа уже не эквивалентна исходной.

В двумерном алгоритме Гаусса–Зейделя обновление $a_{i,j}$ на новом временном слое использует четыре соседних элемента: $a_{i-1,j}$, $a_{i+1,j}$, $a_{i,j-1}$ и $a_{i,j+1}$. При пересчете значения $a_{i-1,j}$ и $a_{i,j-1}$ уже относятся к новому слою, а два других соседа — к предыдущему. Для такого шаблона получаются векторы $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$, $(1, -1, 0)$, $(1, 0, -1)$. Отрицательные компоненты показывают, что перед тайлингом по пространственным координатам нужно выполнить скашивание.

2.3 Алгоритм построения матрицы скашивания

Отрицательные координаты устраняются с помощью автоматически построенной матрицы скашивания. Идея состоит в управляемом смещении внутреннего пространства итераций относительно внешних циклов.

Для каждой дуги зависимости с отрицательной компонентой $\text{dist}_m < 0$ (где $m \in \{1, \dots, n\}$) выполняется элементарное скашивание циклов-носителей этой зависимости относительно цикла m . Параметр скашивания f выбирается равным модулю отрицательной координаты: $f = |\text{dist}_m|$. Каждое такое элементарное скашивание задается нижнетреугольной матрицей s , в которой на главной диагонали расположены единицы, а элемент $s_{k,m} = f$ фиксирует смещение цикла-носителя k относительно цикла m .

Если дуга имеет несколько отрицательных координат, то формируется композиция матриц. Для всего множества дуг графа зависимостей компилятор собирает множество матриц единичных скашиваний S . Результирующая глобальная матрица скашивания skew вычисляется путем покомпонентного выбора максимальных элементов из всех матриц множества S :

$$\text{skew}_{i,j} = \max_{s \in S} (s_{i,j}). \quad (2)$$

Умножение исходного вектора счетчиков на матрицу *skew* переводит вычисления в новое пространство итераций. В этом пространстве отрицательные компоненты векторов расстояний отсутствуют, что дает достаточное условие для корректного многомерного тайлинга.

Корректность этой процедуры опирается на три свойства. Матрица *skew* по построению нижнетреугольная и имеет единицы на главной диагонали, поэтому соответствующее отображение целочисленного пространства итераций биективно. Покомпонентный максимум по матрицам единичных скашиваний сохраняет неотрицательность для каждой отдельной дуги. Наконец, первая координата вектора расстояний, соответствующая временному циклу, остается равной единице, а значит порядок временных слоев не меняется.

Для генерации кода одной матрицы недостаточно: нужен порядок элементарных скашиваний. Преобразования из одной строки, если они независимы, можно переставлять. Поэтому синтезированная матрица задает не только математическое отображение, но и порядок действий для генератора кода.

2.4 Блочное разбиение и оптимизация порядка обхода внутри тайла

После скашивания пространство итераций разбивается ортогональными гиперплоскостями на многомерные блоки с размерами $d_1, d_2, \dots, d_n$. Этот этап нужен прежде всего для временной локальности: данные, загруженные при обработке одного блока, должны использоваться повторно до вытеснения из кэша.

Классический тайлинг можно дополнить изменением обхода внутри блока. Для этого наиболее глубоко вложенный цикл среди циклов обхода тайла переставляется наружу. Геометрически обход переходит от горизонтальных сечений к сечениям, параллельным боковой грани тайла.

Эффект такого изменения хорошо виден по балансу чтений. Обозначим чтение из оперативной памяти через M , из кэш-памяти L1 через C , а из регистров через R . Для алгоритма Гаусса–Зейделя обычный обход по горизонтальным сечениям дает для большинства внутренних точек блока одно чтение из регистров и три чтения из кэша ($1R + 3C$).

При обходе по боковым сечениям для внутренних точек получается $2R + 2C$. Доля обращений к регистрам увеличивается, нагрузка на L1-кэш снижается, а данные реже вытесняются до повторного использования. В статье Метелицы с этим

изменением порядка обхода связывается дополнительное ускорение относительно классического варианта полиэдральной оптимизации в PLUTO [10].

Эквивалентность сохраняется, поскольку речь идет о перестановке двух смежных циклов внутри фиксированного тайла. Лексикографический порядок векторов зависимостей уже приведен к неотрицательному виду на этапе скашивания, поэтому такая перестановка его не нарушает.

2.5 Аналитическая модель выбора оптимальных размеров тайлов

Параметры блочного разбиения компилятор может выбирать по аналитической модели. Она связывает геометрию блока с характеристиками целевого процессора и оценивает объем памяти V , необходимый для элементов массива, которые активно переиспользуются при вычислении одного шага внутри тайла.

Для двумерного расчетного шаблона минимизация промахов в L1-кэше требует, чтобы совокупный рабочий набор данных трех смежных сечений тайла непрерывно и целиком помещался в кэш-памяти. Обозначим физическую емкость кэша L1, доступную одному ядру и выраженную в элементах массива, как $|L_1|$. Количество уникальных элементов массива, вовлеченных в вычисление одного внутреннего сечения с учетом краевых точек шаблона, оценивается как:

$$V \leq d_1 d_2 + 2(d_1 + d_2) = (d_1 + 2)(d_2 + 2) - 4. \quad (3)$$

Отсюда вытекает физическое ограничение для оптимизирующего компилятора:

$$(d_1 + 2)(d_2 + 2) - 4 \leq |L_1|. \quad (4)$$

Математически известно, что при фиксированной площади сечения ($d_1 \times d_2$) его периметр, который отвечает за подкачку граничных значений из более медленных уровней памяти, достигает минимума в форме квадрата, то есть при $d_1 = d_2$. Когда компилятор разрешает неравенство относительно симметричного тайла, он получает аналитическое выражение для теоретически оптимального размера:

$$d_1 = d_2 \approx \sqrt{|L_1| + 4} - 2. \quad (5)$$

Для L1-кэша объемом 32 Кбайт, вмещающего 4096 восьмибайтовых чисел с плавающей точкой двойной точности (тип REAL(8)), оценка дает $d_1 = d_2 \approx \sqrt{4100} - 2 \approx 62$. Эксперименты показывают, что максимальное ускорение на многоядерных архитектурах действительно достигается вблизи таких размеров. Поэтому аналитическая модель пригодна как основа для автоматической настройки параметров компилятора [11].

На практике модель приходится корректировать. Часть кэша занята метаданными и соседними структурами; ассоциативность может вызывать конфликтные промахи; кроме того, кэш иногда совместно используется несколькими ядрами. Поэтому размер тайла обычно уменьшают на 10–20% относительно теоретической оценки. Такую поправку можно подобрать автоматически небольшой серией запусков на целевой машине.

В трехмерном случае аналитическая модель приводит к кубическому уравнению, а оптимальный размер оценивается как $d \approx \sqrt[3]{|L_1|} - 2$. Для кэша объемом 32 Кбайт это около 14 элементов по каждому измерению, то есть заметно меньше двумерного варианта при том же общем объеме тайла. Причина естественна: с ростом размерности размер блока по отдельной координате уменьшается, а отношение полезного объема к поверхности ухудшается. Поэтому для трехмерных и более сложных задач требуется многоуровневый тайлинг: внешний уровень работает на локальность в L2 или L3, внутренний — на L1 и регистровый файл.

Если параметры целевой машины заранее неизвестны, размер тайла можно выбирать позднее. Один вариант — сгенерировать несколько версий кода с разными размерами блоков и выбрать подходящую во время исполнения по результатам профилирования. Другой вариант — оставить размер тайла параметром функции и определить его после короткой калибровочной серии измерений. Оба способа дают переносимую производительность без ручной перекомпиляции под каждую аппаратную платформу.

2.6 Дополнительные скалярные оптимизации линейных участков

Для того чтобы раскрыть потенциал оптимизированного кода, глобальные цикловые преобразования пространства итераций должны сопровождаться локальными скалярными оптимизациями в рамках сгенерированных блоков [12].

Изменение топологии гнезда циклов усложняет математическую структуру индексных выражений и добавляет в них дополнительные арифметические операции. В тела циклов внедряются конструкции, которые включают умножения счетчиков тайлов на константы d_i , вычисления минимальных или максимальных границ, а также алгебраические смещения. Для того чтобы уменьшить накладные расходы от этих вычислений, компилятор применяет:

- вынесение инвариантных выражений: компилятор идентифицирует и выносит за пределы заголовка цикла подвыражения, операнды которых константны в рамках конкретного цикла; это предотвращает их многократное перевычисление на каждой итерации;
- линеаризацию и алгебраическое упрощение выражений: многомерные аффинные индексные выражения приводятся к каноническому, упрощенному виду; медленные целочисленные умножения заменяются индуктивными сложениями и сдвигами;
- оптимизацию заголовка цикла: границы цикла, которые для тайлинга обычно представляют собой выражения вида $\min(d_1 \cdot (k + 1), N)$, подвергаются предварительному анализу; компилятор генерирует отдельные эпилог и пролог для краевых случаев, что позволяет избавиться от проверок минимума внутри часто выполняемого участка;
- оптимизацию вычисления указателей массивов: в Фортране адрес элемента массива вычисляется по формуле линеаризации индексов, и если эта формула содержит инвариантные относительно внутреннего цикла слагаемые, то компилятор сводит их в единый указатель, который наращивается постоянным шагом.

Сочетание глобальной реорганизации пространства итераций с локальной оптимизацией скалярных операций позволяет получить дополнительное ускорение (помимо распараллеливания) в 1,5–1,8 раза.

ЗАКЛЮЧЕНИЕ

В работе рассмотрены теоретические основы оптимизации многомерных вложенных циклов для компилятора подмножества языка Фортран. Для регулярных гнезд циклов основными элементами анализа выступают пространство итераций, информационные зависимости и лексикографическая положительность векторов расстояний после преобразований.

Для гнезд циклов итерационного типа обоснована последовательность преобразований, включающая построение матрицы скашивания, блочное разбиение, изменение порядка обхода точек внутри блока и распараллеливание тайлов методом гиперплоскостей. Такая схема сохраняет семантику исходной программы и позволяет использовать параллелизм на уровне крупных вычислительных блоков, что особенно важно для алгоритмов с зависимостями между временными слоями.

Отдельно рассмотрена аналитическая модель выбора размеров тайлов. Она связывает параметры блочного разбиения с емкостью кэш-памяти и объясняет, почему близкие к квадратным сечения блоков обеспечивают более выгодное соотношение между объемом полезных вычислений и количеством граничных обращений к памяти. Дополнительные скалярные оптимизации, такие как вынесение инвариантных выражений и линеаризация индексных выражений, уменьшают накладные расходы, возникающие после структурных преобразований.

Эти результаты применимы при проектировании оптимизирующего компилятора подмножества Фортрана. Транслятор может выполнять часть адаптации вычислительных программ к иерархии памяти и многоядерной архитектуре, сохраняя возможность формальной проверки корректности преобразований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Allen R., Kennedy K.* Optimizing Compilers for Modern Architectures: A Dependence-based Approach. — San Francisco : Morgan Kaufmann, 2002.
2. *Cooper K. D., Torczon L.* Engineering a Compiler. — 2nd. — Burlington, MA : Morgan Kaufmann, 2011.
3. Компиляторы: принципы, технологии и инструментарий / А. В. Ахо [и др.]. — 2-е изд. — М. : Вильямс, 2008.
4. *Banerjee U.* Dependence Analysis for Supercomputing. — Boston : Kluwer Academic Publishers, 1988.
5. *Feautrier P.* Some efficient solutions to the affine scheduling problem. I. One-dimensional time // International Journal of Parallel Programming. — 1992. — Т. 21, № 5. — С. 313—347.
6. *Quillere F., Rajopadhye S., Wilde D.* Generation of efficient nested loops from polyhedra // International Journal of Parallel Programming. — 2000. — Т. 28, № 5. — С. 469—498.
7. *Wolf M. E., Lam M. S.* A loop transformation theory and an algorithm to maximize parallelism // IEEE Transactions on Parallel and Distributed Systems. — 1991. — Т. 2, № 4. — С. 452—471. — DOI: 10.1109/71.97902.
8. *Lamport L.* The parallel execution of DO loops // Communications of the ACM. — 1974. — Т. 17, № 2. — С. 83—93. — DOI: 10.1145/360827.360844.
9. *Метелица Е. А.* Обоснование методов ускорения гнёзд циклов итерационного типа // Программные системы: теория и приложения. — 2024. — Т. 15, 1(60). — С. 63—94. — DOI: 10.25209/2079-3316-2024-15-1-63-94. — URL: <https://www.mathnet.ru/rus/ps440>.
10. A practical automatic polyhedral parallelizer and locality optimizer / U. Bondhugula [и др.] // Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI). — 2008. — С. 101—113. — DOI: 10.1145/1375581.1375595.

11. *Метелица Е. А.* Автоматизация распараллеливания программ со сложными информационными зависимостями : дис. . . . канд. / Метелица Е. А. — М. : ИПМ им. М. В. Келдыша РАН, 2024. — URL: <https://keldysh.ru/council/1/2024-metelitsa/org.pdf>.
12. *Метелица Е. А.* Об ускоряющих преобразованиях программ для решения обобщённой задачи Дирихле // Вычислительные методы и программирование. — 2024. — Т. 25, № 3. — С. 292—301.